

Seminar 7: EXCEPTIONS

- Definition
- Types of exceptions
- When they are invoked
- Handling exceptions

Def: An exception is a PL/SQL identifier that is associated with an abnormal situation raised during the execution of a block. Raising an exception has the effect of terminating the block. The commands that follow raising the exception are ignored. The exception can be handled in the Exception section of a PL/SQL block.

An exception can be raised in two ways:

- An Oracle error appears and the exception associated with it is automatically invoked
- The exception is explicitly invoked by using RAISE or RAISE_APPLICATION_ERROR instructions.

Exception types

Type	Way of handling
Pre-defined exceptions associated with errors that usually are met in PL/SQL blocks	They don't have to be declared, Oracle server automatically raises them, but they have to be handled in the EXCEPTION section.
Oracle non-predefined exceptions, that are handled by the user using their error code	They have to be declared in the declaring section. Oracle server automatically raises them, but they have to be handled in the EXCEPTION section.
User-defined exceptions that are associated with certain processing conditions	They have to be declared in the declaring section, are raised by the user and handled in the Exception section.

EXCEPTIONS:

- Declaration (yes/no)
- Raising (automatically/RAISE)
- Handling

Handling all exceptions is made in the EXCEPTION section of PL/SQL blocks, this way:

EXCEPTION

```
WHEN exception1 [OR exception2 ...] THEN  
    statement1 ;  
    statement2 ;  
    ...
```

```
[WHEN exception3 [OR exception4 ...] THEN  
    statement1 ;  
    statement2 ;  
    ...]  
[WHEN OTHERS THEN  
    statement1 ;  
    statement2 ;  
    ...]
```

1. Oracle pre-defined exceptions

Name	Error number	Description
NO_DATA_FOUND	ORA-01403	A SELECT statement that should have returned a row didn't return any.
TOO_MANY_ROWS	ORA-01422	A SELECT statement that should have returned a row, returned many rows instead.
INVALID_CURSOR and CURSOR_ALREADY_OPEN	ORA-01001	Errors associated with cursors (for example, trying to open a cursor that is already opened).

```
DECLARE...  
BEGIN...  
EXCEPTION  
WHEN exception_name THEN statements;  
END;
```

Example: Display the product 13. If there is no such product, handle the exception.

```
DECLARE  
name VARCHAR2(20);  
BEGIN  
SELECT product_name INTO name  
FROM product_information  
WHERE product_id=13;  
dbms_output.put_line(name);  
EXCEPTION  
WHEN NO_DATA_FOUND THEN  
dbms_output.put_line('The product doesn't exist!');  
END;
```

2. Oracle non-predefined exceptions

```
DECLARE
Exception_name EXCEPTION;
PRAGMA EXCEPTION_INIT(exception_name, code);
....
BEGIN
....
EXCEPTION
WHEN exception_name THEN statements;
END;
```

A non-predefined exception can be handled by giving it a name or by using *WHEN OTHERS* clause. In PL/SQL, the directive *EXCEPTION_INIT* makes the compiler to associate a name with a number (code) of an exception. This allows further on, to address the exception by its name and to write its handler.

Managing this type of exceptions is made through the following steps:

- 1) Declaring the exception: is made in the DECLARE area of a block
Exception_name EXCEPTION;
- 2) Associating the error code with the previously declared exception: is made in the DECLARE area by using *EXCEPTION_INIT* compilation directive:
PRAGMA EXCEPTION_INIT(exception_name, error_code);
Error_code is a standard Oracle error code.
- 3) Handling the exception in EXCEPTION area of a block:
EXCEPTION
WHEN exception_name THEN ...;

There are two attributes that can be used for handling exceptions:

SQLCODE – returns the numeric code of the error. It can have the following values:

- 0 – no error;
- 1 – a user-defined error that has no code associated with it;
- +100 – NO_DATA_FOUND exception;
- Negative number – error code;

SQLERRM – returns the exceptions' message.

Example: Insert a row in the Locations table. For the location_id use the value 213 and for the country_id use NULL. An exception having the code ORA-01400 will be automatically raised. Handle the exception regarding violating the integrity constraint.

```
DECLARE
INSERT_EXCEPT EXCEPTION;
PRAGMA EXCEPTION_INIT(INSERT_EXCEPT, -01400);
BEGIN
insert into locations (location_id, country_id) values ('213', NULL);
EXCEPTION
WHEN insert_except THEN
DBMS_OUTPUT.PUT_LINE('country_id cannot be null');
```

```
DBMS_OUTPUT.PUT_LINE(SQLERRM);  
END;
```

3. User-defined exceptions

```
DECLARE  
Exception_name EXCEPTION;  
...  
BEGIN  
...  
RAISE exception_name;  
....  
EXCEPTION  
WHEN exception_name THEN statements;  
END;
```

Or if you want to have a code and a message for the exception:

```
DECLARE  
Exception_name EXCEPTION;  
PRAGMA EXCEPTION_INIT(exception_name, code);  
...  
BEGIN  
...  
RAISE_APPLICATION_ERROR(code,message);  
....  
EXCEPTION  
WHEN exception_name THEN statements;  
END;
```

Example: Raise an error if the user tries to execute the PL/SQL block before 5 o'clock pm.

```
DECLARE  
e_exc1 EXCEPTION;  
  
BEGIN  
IF TO_NUMBER(TO_CHAR(SYSDATE, 'HH24'))<=17 THEN  
RAISE e_exc1;  
END IF;  
  
EXCEPTION  
WHEN e_exc1 THEN  
dbms_output.put_line('It is '||TO_CHAR(SYSDATE, 'HH24'));  
dbms_output.put_line('Operation not yet allowed');  
END;
```

User-defined exceptions can also be handled as Oracle exceptions, by giving them error codes and error messages. In this case, the error code can have values between -20999 and -20000.

```
DECLARE
e_exc1 EXCEPTION;
PRAGMA EXCEPTION_INIT(e_exc1, -20600);
BEGIN
IF TO_NUMBER(TO_CHAR(SYSDATE, 'HH24'))<=17 THEN
RAISE_application_error(-20600,'error message');
END IF;

EXCEPTION
WHEN e_exc1 THEN
dbms_output.put_line('It is '||TO_CHAR(SYSDATE, 'HH24'));
dbms_output.put_line('Operation not yet allowed');
dbms_output.put_line(sqlerrm);
END;
```

The exceptions can have associated:

- name
- message
- code

By default, some exceptions have these attributes set, while others don't. See the following table:

Exception	name	code	message
Oracle Pre-defined	v	v	v
Oracle Non-pre-defined	x	v	v
User defined	x	x	x

Exceptions' propagation

Once an exception has been raised, the system will search for its handling in all the outer blocks until a proper handler is found. If the exception is handled by a block (or subprogram) the execution ends normally, otherwise it ends with error.

Check the following examples and their results:

```
<<block1>>
declare
exc exception;
var number;
begin
var:=6/0;
<<block2>>
begin
```

```
dbms_output.put_line('message in block2');  
exception when others then  
dbms_output.put_line(sqlerrm);  
end block2;  
raise exc;  
dbms_output.put_line('message in block1');  
exception when exc then  
dbms_output.put_line('an exception!');  
end block1;  
Result: unhandled error
```

```
<<block1>>  
declare  
exc exception;  
var number;  
begin  
var:=6/0;  
<<block2>>  
begin  
dbms_output.put_line('message in block2');  
exception when others then  
dbms_output.put_line(sqlerrm);  
end block2;  
raise exc;  
dbms_output.put_line('message in block1');  
exception when exc then  
dbms_output.put_line('an exception!');  
when others then  
dbms_output.put_line(sqlerrm);  
end block1;  
Result: ORA-01476: divisor is equal to zero
```

```
<<block1>>  
declare  
exc exception;  
var number;  
begin  
  
<<block2>>  
begin  
dbms_output.put_line('message in block2');  
raise exc;  
end block2;  
  
var:=6/0;  
dbms_output.put_line('message in block1');
```

```
exception  
when exc then  
dbms_output.put_line('an exception!');  
when zero_divide then  
dbms_output.put_line(sqlerrm);  
end block1;
```

Result:
message in block2
an exception!

Example of pre-defined exceptions:

- **no_data_found**—Single row SELECT returned no data.
- **too_many_rows**—Single row SELECT returned more than one row.
- **invalid_cursor**—Illegal cursor operation was attempted.
- **value_error**—Arithmetic, conversion, truncation, or constraint error occurred.
- **invalid_number**—Conversion of a number to a character string failed.
- **zero_divide**—Attempted to divide by zero. s